LABORATORUL 3

Tabla Snake

Astăzi desenăm o tablă statică. Mișcarea începe la Lab 4.

- Reprezentăm pozițiile prin celule, desenăm grila și șarpele și identificăm celula atinsă.
- Datele jocului rămân separate de desenare.

2 Legătura cu cursul

Curs 2, p. 16–18: View, ViewGroup și interfața Android.

- Curs 3, p. 5–8: procese, pornirea aplicațiilor și Zygote.
- Canvas, Paint și onDraw() sunt completări practice introduse în acest laborator.

RecyclerView și Fragments din Cursul 3 vor fi aplicate la Lab 7–8.

3 Un View personalizat

Un View ocupă o zonă a interfeței. Printr-o subclasă definim desenarea și interacțiunea proprii.

```
public class BoardView extends View {
    // Constructori și metode oferite în starter
}
```

- *GameActivity găzduiește ecranul. BoardView reprezintă tabla din acel ecran. GameState păstrează datele.*

4 Cum se leagă fișierele

GameActivity.java	încarcă activity_game.xml
activity_game.xml	conține BoardView
BoardView.java	desenează tabla și primește touch-ul
GameState.java	păstrează șarpele și obstacolele
strings.xml	păstrează textele aplicației

Fiecare fișier are o singură responsabilitate.

5 Canvas și Paint

Canvas oferă operații de desenare; Paint stabilește culoarea, stilul și grosimea.

```
canvas.drawRect(20f, 20f, 100f, 100f, snakePaint);
```

- Parametrii sunt left, top, right, bottom și Paint. Ultimele coordonate nu sunt lățimea și înălțimea.

● *Pregătim Paint o singură dată și îl refolosim.*

? **Ce dimensiune are dreptunghiul de mai sus: 100 × 100 sau 80 × 80?**

6 Răspuns: Canvas și Paint

canvas.drawRect(20f, 20f, 100f, 100f, snakePaint): ce dimensiune are dreptunghiul, 100 × 100 sau 80 × 80?

- 80 × 80.
- Lățimea = right − left = 100 − 20 = 80.
- Înălțimea = bottom − top = 100 − 20 = 80.

drawRect() primește colțurile dreptunghiului, nu lățimea și înălțimea.

7 Coordonatele tablei

În View, originea este stânga-sus, x crește spre dreapta și y în jos.

- În joc folosim (col, row), cu indici de la zero.

```
Cell head = new Cell(3, 2);
```

- Capul este în coloana a patra, rândul al treilea.

Pe o tablă 10 × 10, indicii valizi sunt 0–9 pe fiecare axă.

8 Dimensiunea și încadrarea tablei

onSizeChanged() este apelată când View-ul primește sau își schimbă dimensiunea. Aici calculăm geometria tablei.

```
cellSize = Math.min(availableWidth / COLS,
                    availableHeight / ROWS);
boardWidth = COLS * cellSize;
boardHeight = ROWS * cellSize;
```

- *Dimensiunile disponibile sunt float, după scăderea padding-ului. boardLeft și boardTop reprezintă originea tablei centrate.*

? **Pentru 300 × 200 px disponibili și 10 × 10 celule, cât este cellSize și cât ocupă tabla?**

9 Răspuns: Dimensiunea și încadrarea tablei

Pentru 300 × 200 px disponibili și 10 × 10 celule, cât este cellSize și cât ocupă tabla?

- cellSize = min(300 / 10, 200 / 10) = min(30, 20) = 20 px.
- Tabla ocupă 10 × 20 = 200 px pe fiecare axă: 200 × 200 px.
- Pe orizontală rămân 100 px liberi, împărțiți egal: boardLeft = 50.

Alegem minimul ca celulele să fie pătrate și tabla să încapă.

10 Starea jocului

Starea descrie situația curentă: pozițiile șarpelui și obstacolelor.

```
List<Cell> snake = new ArrayList<>(Arrays.asList(
    new Cell(4, 4), new Cell(3, 4), new Cell(2, 4)));
```

- Cell păstrează col și row. GameState nu are nevoie de Canvas sau Paint.

Folosim o listă modificabilă pentru a o continua cu add/remove la Lab 4.

11 Rolul metodei onDraw

Android apelează onDraw() când View-ul trebuie desenat.

```
@Override
protected void onDraw(Canvas canvas) {
    super.onDraw(canvas);
    if (cellSize <= 0f) return;
    drawGrid(canvas);
    drawSnake(canvas);
    drawObstacles(canvas);
}
```

- *Metoda citește starea și o desenează. Nu mută șarpele și nu modifică scorul.*

? **Dacă sistemul redesenează de două ori aceeași stare, trebuie să avanseze șarpele?**

12 Răspuns: Rolul metodei onDraw

Dacă sistemul redesenează de două ori aceeași stare, trebuie să avanseze șarpele?

- Nu.
- onDraw() poate fi apelată de mai multe ori pentru aceeași stare.
- Șarpele avansează doar când se schimbă starea jocului (de la Lab 4).

onDraw() afișează starea, nu o modifică.

13 Desenarea grilei

O tablă cu COLS coloane are COLS + 1 margini verticale.

```
for (int col = 0; col <= COLS; col++) {
    float x = Math.round(boardLeft + col * cellSize);
    canvas.drawLine(x, top, x, bottom, gridPaint);
}
```

- *top și bottom sunt marginile de sus și de jos ale tablei. Math.round() evită coordonatele fracționare și stabilizează poziția liniilor. Pentru liniile orizontale iterăm rândurile și calculăm y.*

14

Două sisteme de coordonate

Joc	Ecran
(col, row)	(x, y) în pixeli
poziție logică	poziție vizuală
Cell(2, 1)	dreptunghiul desenat

- *Desenare: Cell → pixeli. Touch: pixeli → Cell.*

15 De la celulă la dreptunghi

Transformăm coordonatele logice în coordonate locale ale View-ului.

```
float left = boardLeft + cell.col * cellSize;
float top = boardTop + cell.row * cellSize;
canvas.drawRect(left, top,
    left + cellSize, top + cellSize, paint);
```

- *drawCell() aplică formula pentru obstacole. Prima variantă a șarpelui, drawSnakeSimple(), folosește aceeași formulă pentru fiecare segment.*

? **Cu boardLeft=20, boardTop=40 și cellSize=30, unde începe celula (2,1)?**

16 Răspuns: De la celulă la dreptunghi

Cu boardLeft=20, boardTop=40 și cellSize=30, unde începe celula (2,1)?

- left = 20 + 2 × 30 = 80
- top = 40 + 1 × 30 = 70
- Celula (2,1) începe la (80, 70) și se termină la (110, 100).

Coloana dă x, rândul dă y.

17 De la atingere la celulă

MotionEvent descrie atingerea; getX()/getY() sunt coordonate locale View-ului.

- Verificăm mai întâi interiorul tablei, cu limita dreaptă și inferioară excluse. Apoi:

```
int col = (int) ((x - boardLeft) / cellSize);
int row = (int) ((y - boardTop) / cellSize);
```

- Cu originea (20,40), celula de 30 px și atingerea (95,85), rezultă celula (2,1).

- *cellFromTouch() întoarce null în afara tablei.*

? **De ce verificăm interiorul înainte de conversia la int?**

18 Răspuns: De la atingere la celulă

De ce verificăm interiorul înainte de conversia la int?

- Conversia la int taie zecimalele spre zero.
- O atingere puțin în stânga tablei dă −0.5, iar (int) −0.5 = 0, deci ar fi acceptată greșit coloana 0.
- O atingere exact pe marginea dreaptă ar da col = COLS, în afara tablei.

Întâi verificăm că atingerea e în tablă, apoi calculăm celula.

19

Integrarea touch-ului

onTouchEvent() primește evenimentele gestului. În BoardView tratăm ACTION_DOWN, ACTION_UP și ACTION_CANCEL.

```
Cell cell = cellFromTouch(event.getX(), event.getY());
if (cell != null) {
    Log.d("Lab03", "col=" + cell.col + ", row=" + cell.row);
} else {
    Log.d("Lab03", "atingere in afara tablei");
}
```

- La ACTION_DOWN aflăm celula, iar la ACTION_UP apelăm performClick(), pe care BoardView o suprascrie (pentru accesibilitate). return true arată că View-ul preia evenimentul.

Atingerea identifică o celulă; nu mișcă încă șarpele.

20 Alternativa cu Views

O tablă mică poate fi construită din celule View, de exemplu într-un GridLayout.

- Canvas: desenăm piesele și calculăm celula atinsă.
- Views: fiecare celulă are aspect și listener propriu.
- Memory și Tic-Tac-Toe pot folosi Views. Snake și Pong pot folosi Canvas.

Imaginile pot fi folosite în ambele variante.

21 Procesul aplicației și Zygote

Procesul are memorie și resurse proprii. În fluxul obișnuit, Android folosește Zygote pentru crearea proceselor aplicațiilor.

- Linux gestionează procesele și firele de execuție.

```
Log.d(TAG, getClass().getSimpleName() + " PID=" + android.os.Process.myPid());
```

- În terminal: adb shell pidof NUME_PACHET

Înlocuim NUME_PACHET cu identificatorul real. PID-ul identifică procesul; nu este UID-ul aplicației.

? Meniul și jocul afișează același PID. Este o problemă?

22 Răspuns: Procesul aplicației și Zygote

Meniul și jocul afișează același PID. Este o problemă?

- Nu.
- Activitățile aplicației rulează, de obicei, în același proces.
- În Logcat: MainActivity PID=12345 și GameActivity PID=12345.

O activitate nu este un proces. PID-ul se schimbă doar dacă procesul aplicației este oprit și pornit din nou.

23 Verificări și diagnostic

Încadrare	tabla complet vizibilă, celule pătrate, margini de sistem respectate, inclusiv după rotație
Atingere cunoscută	alegem celula (2,1), atingem centrul și comparăm Logcat
Poziție inversată	verificăm col versus row
Atingere deplasată	verificăm scăderea boardLeft/boardTop
Celulă în afara tablei	verificăm 0 ≤ col < COLS și 0 ≤ row < ROWS